

LithosAnanke Kernel and Capsule System

Volume II of the StarshipOS Technical Reference

Robert A. James

Contents

1	LithosAnanke Kernel Architecture	5
1.1	Design Philosophy	5
1.2	Boot Sequence	5
1.3	Milestone Status	5
1.4	Memory Layout	5
1.5	Source Tree	5
1.6	Multi-Architecture Support	5
1.7	Experimental Campaign Notes	6
1.8	StarForth, LithosAnanke, and StarshipOS: A Technical Overview	6
1.8.1	StarForth	6
1.8.2	LithosAnanke	8
1.8.3	StarshipOS	9
2	Capsule System	11
2.1	Capsule Types	11
2.2	Content Addressing and Immutability	11
2.3	Birth Protocol	11
2.4	Capsule Descriptor	12
2.5	Memory Layout	12
2.6	Capsule Files	13
2.7	Mama Forth Supervisor	13
3	Word-Level ACL and Security	15
3.1	C Infrastructure	15
3.1.1	Interpreter Hook: Two-Level Check	15
3.2	Three Enforcement Dispositions	16
3.2.1	Adaptive TTL	16
3.2.2	The Pin Ratchet	16
3.3	Inheritance at Birth	16
3.4	ACL.4th: Pure-FORTH Policy	17
3.5	Two-Console Architecture	17
3.5.1	Emergency Console	17
3.5.2	Zuse Console	18
3.6	Zuse Superuser	18
3.6.1	CA Root	18
3.7	Security Model	18
3.8	ACL System in Kernel Context	18
3.9	Formal Verification	19
3.10	Implementation Phases	19
3.11	Future: PKI and Ed25519 (Phase 8)	19

4	Platform Support	21
4.1	Platform Abstraction Layer	21
4.1.1	Platform API	21
4.2	Linux / Hosted	22
4.3	L4Re / Fiasco.OC	22
4.3.1	Package Structure	22
4.3.2	Memory Management via Dataspaces	22
4.3.3	IPC Interface	23
4.3.4	Multi-VM Architecture on L4Re	23
4.3.5	L4Re Timing Backend	23
4.4	Bare Metal: amd64	23
4.5	Bare Metal: aarch64 and riscv64	24
4.6	Raspberry Pi Build	24
4.7	Future Platforms	24
5	L8 Jacquard Mode Selector	25
5.1	Six Operating Modes	25
5.2	Attractor Bucket Mechanism	25
5.3	ssm_jacquard.c Implementation	25
5.4	Jacquard Capsule Variants	25
6	QEMU Testing and Regression Baseline	27
6.1	Running the Kernel in QEMU	27
6.2	Regression Baseline	27

Chapter 1

LithosAnanke Kernel Architecture

1.1 Design Philosophy

1.2 Boot Sequence

The boot sequence from firmware to FORTH REPL:

1. UEFI Firmware → `uefi_loader.c` (BOOTX64.EFI)
2. `ExitBootServices()` — kernel owns hardware; receives `BootInfo{memory map, ACPI, framebuffer}`
3. `kernel_main()` — M1 through M7 milestone initialization:
 - M1 Console init (UART 16550 + framebuffer)
 - M2 PMM — physical memory manager, bitmap allocator
 - M3 VMM — 4-level x86_64 paging
 - M4 IDT + APIC interrupts
 - M5 TSC + HPET + APIC timer (100 Hz heartbeat)
 - M6 `kmalloc` heap (16 MB)
 - M7 StarForth VM bootstrap + capsule loading → ok REPL

1.3 Milestone Status

Table 1.1: LithosAnanke v1.5.3 milestone status

Milestone	Description	Status
M0–M5	Boot, console, PMM, VMM, IDT/APIC, timer	Complete
M6	<code>kmalloc</code> infrastructure	Present (full validation deferred)
M7	StarForth VM bootstrap + capsule execution pipeline	In progress

1.4 Memory Layout

1.5 Source Tree

1.6 Multi-Architecture Support

Table 1.2: Kernel memory layout

Region	Description
Kernel heap	16 MB (<code>kmalloc</code>)
Block RAM LBN 0–991	1 MB dedicated RAM blocks
Kernel ramdrive LBN 2048–3071	1 MB for capsule loading
LBN 2048	Entry point for <code>init.4th</code>

Listing 1.1: StarKernel build targets

```

1 make -f Makefile.starkernel          # Build for amd64
2 make -f Makefile.starkernel ARCH=aarch64
3 make -f Makefile.starkernel ARCH=riscv64
4 make -f Makefile.starkernel qemu      # Run in QEMU with OVMF

```

1.7 Experimental Campaign Notes

1.8 StarForth, LithosAnanke, and StarshipOS: A Technical Overview

Author: Robert A. James. Date: 9 April 2026. Status: Working Document.

1.8.1 StarForth

What It Is

StarForth is a pure C99 FORTH-79 virtual machine with no libc dependencies. It is self-adaptive: it observes its own execution behavior and continuously adjusts internal parameters to maintain a conservation invariant called $K \equiv 1.0$. It runs on x86_64, aarch64, and RISC-V.

Why FORTH

FORTH is correct for this work for reasons beyond familiarity. FORTH words are discrete, classifiable, observable computational units — each word has a definite stack-effect signature, each word consumes a known quantity of computational energy, each word is a particle with measurable properties. This is the foundation of StarForth’s self-adaptive behavior.

The Primitive Taxonomy

StarForth implements 23 word modules (18 FORTH-79 standard, 5 StarForth extensions). Every primitive word carries quantum-analog properties derived from its stack-effect signature:

Spin. Derived from net stack-effect arithmetic. A word consuming two values and producing one has spin $-\frac{1}{2}$; these values fall directly from (`n n - n`) notation, as half-integer spin falls from the Dirac equation.

Charge. The net stack delta. Charge conservation across a complete program means the program is stack-balanced.

Heat. Execution frequency and recency. Heat decays over time, analogous to radioactive decay.

Mass. Bytecode footprint of the word definition.

Color. Binding state: white words are fully resolved; colored words carry unresolved forward references.

Three of these quantities — spin, charge, and color — behave as genuine conservation laws across well-formed programs. This is a structural property of the system discovered empirically and subsequently formalized in Isabelle/HOL.

The SSM and James Law

The SSM (Steady State Machine) is the self-adaptive core of StarForth. It monitors execution behavior through instrumented subsystems and continuously adjusts internal parameters to maintain $K \equiv 1.0$.

James Law ($K \equiv 1.0$) is a conservation invariant: the normalized total execution energy of the StarForth universe is conserved at unity across all configurations and workloads. Validation basis: 38,400 experimental runs spanning 128 factorial parameter configurations, CV below 1%, published on SSRN (abstract 5930134).

The SSM operates through four mechanisms:

Rolling Window of Truth (RWOT). A sliding observation window encoding five observables: word identity, $\text{prev} \rightarrow \text{word}$ transition, instruction pointer position, execution heat, and causal stack depth. Updated at heartbeat tick intervals, making it a coarse-grained lattice observable.

Adaptive Heartbeat. A self-adjusting timer governing when the SSM evaluates system state and adjusts parameters. The system’s intrinsic rhythm.

Physics Hooks. Every word execution in the inner interpreter loop is bracketed by `physics_pre_execute` and `physics_post_execute` calls, updating heat, decay, and window measurements.

Decay. Execution heat decays between heartbeat ticks, keeping the observable window relevant to current rather than historical behavior.

The Phase Space and Attractor

Six workload types have been studied experimentally: STABLE, VOLATILE, TRANSITION, TEMPORAL, DIVERSE, and OMNI. Each produces a distinct trajectory in phase space. All trajectories converge to the same attractor: $K \equiv 1.0$. Phase-space portrait visualizations confirm attractor convergence across all workload types on all three target architectures.

The Manifold Navigation Controller (Theoretical)

Given a bounded system with a known conservation invariant and predictable attractor manifold, a controller can:

1. Identify the system’s current phase-space position using three observables: RWOT (position), adaptive heartbeat metadata (velocity), and spin-class transition rate (acceleration — requires instrumentation).
2. Compute a perturbation vector sufficient to transport the system from its current manifold to an arbitrary target manifold.
3. Inject the perturbation — the burn — and release.
4. Allow the system to coast back to $K \equiv 1.0$ under its own restoring force.

The controller fires once and releases. The conservation invariant guarantees the return journey. This orbital-mechanics model scales from the word level through the VM, OS, FPGA, and prospectively to custom silicon.

1.8.2 LithosAnanke

What It Is

LithosAnanke is a bare-metal operating system for x86_64, currently at milestone M7 (VM integration). It boots via UEFI, manages physical and virtual memory, handles interrupts via IDT/APIC, maintains a timer, manages a heap, and hosts the StarForth VM as its primary computational substrate. Written in C99 with no external dependencies.

The Capsule System

The primary organizational unit is the *capsule*: a content-addressed, 64-byte cache-line-aligned descriptor. A capsule's ID is its content hash. Identity is content: two capsules with identical content have identical IDs. Modification produces a new capsule with a new ID. Capsules are immutable by design; mutation produces new capsules.

Component Architecture

Hera. The foundational VM and heartbeat. The system's ground state.

Hermes. Message entropy and TTL decay. Manages the message-passing fabric, monitors message lifecycle, and reaps expired messages. The natural home of the perturbation controller extension.

Artemis. Persistence and event sourcing. Manages the capsule store, maintains the event log, and handles durable state.

Components communicate exclusively through message passing. There is no shared mutable state between components.

Authentication and Identity

LithosAnanke implements a hardware-rooted authentication model with no usernames and no passwords. Identity is carried on a physical thumb drive containing two raw block partitions.

Access control is governed by temporal ACL grants that decay via the same Hermes TTL mechanism that governs message lifetime. The default state of the permission system is no access. Decay to zero requires no explicit revocation.

Normalized Virtual Block Space

From any VM's perspective, LithosAnanke presents a single contiguous block address space. Physical origin — local drive, system hardware, or remote cloud mount — is transparent. Blocks appear as grants are issued; blocks vanish as grants decay. Security is expressed as address-space geometry.

The Three UX Tiers

CLI. The traditional FORTH terminal interface. Full power, zero overhead. The REPL is operational.

GUI. A wireframe soundstage rendered from rasterized vector graphics. Words are physical objects; the user assembles colon definitions spatially. The FORTH text is always visible underneath.

VR. The full holodeck realization. The user stands inside the phase space. Words have mass, spin, heat, and charge as observable properties. Manifold navigation is literally navigable space.

All three tiers are windows into identical underlying mechanics.

1.8.3 StarshipOS

The SSPU

The SSPU (Steady State Processing Unit) is a novel processor architecture extending the StarForth/SSM work into hardware. It communicates entirely via a message-passing fabric called Gefyra. There are no traditional buses. Targeted for initial FPGA implementation on a Digilent Genesys ZU (Zynq UltraScale+).

Milestone Status

Milestone	Description	Status
M0	UEFI boot	Complete
M1	Physical memory manager	Complete
M2	Virtual memory manager	Complete
M3	IDT/APIC interrupt handling	Complete
M4	Timer	Complete
M5	Heap	Complete
M6	–	Complete
M7	VM integration	Active frontier
–	Manifold Navigation Controller	Theoretical
–	SSPU FPGA implementation	Planned (Genesys ZU)

Design Philosophy

The recurring pattern in this architecture is the elimination of problem domains rather than their simplification. No scheduler, no traditional memory manager, no username/password authentication, no logout ceremony, no coordination layer for distributed state — each elimination is a deliberate architectural decision. The question at each design point was not “how do we implement this?” but “do we need this at all?”

Key Terms

Term	Definition
K $\equiv$ 1.0	James Law — conservation invariant of normalized execution energy
SSM	Steady State Machine — self-adaptive core of StarForth
RWOT	Rolling Window of Truth — coarse-grained phase-space observable
SSPU	Steady State Processing Unit — novel message-passing processor
Gefyra	Message-passing fabric connecting SSPU processing elements
Capsule	Content-addressed, 64-byte aligned immutable descriptor
Hera	Foundational VM and heartbeat component
Hermes	Message entropy, TTL decay, lifecycle management
Artemis	Persistence and event sourcing

Chapter 2

Capsule System

A capsule is an immutable, content-addressed VM initialization payload. The capsule identifier is the XXHash64 hash of the capsule’s content; any mutation of the payload produces a different hash and is therefore detectable at birth time. Capsules provide the mechanism by which new FORTH virtual machine instances are initialized in a verifiable and reproducible manner, whether on a hosted Linux system or within the LithosAnanke bare-metal kernel.

2.1 Capsule Types

Three capsule types exist, distinguished by a single type-code field in the capsule descriptor.

Table 2.1: Capsule types

Type code	Name	Purpose
(m)	MAMA_INIT	Exactly one Mama VM initialization capsule
(p)	PRODUCTION	Truth-bearing baby VM initializers
(e)	EXPERIMENT	DoE workload-only initializers

Exactly one `MAMA_INIT` capsule exists per deployment. It initializes the root VM (Mama Forth) from which all child VMs are subsequently born. `PRODUCTION` capsules carry the authoritative personality for truth-bearing baby VMs. `EXPERIMENT` capsules supply workload-only initialization for Design of Experiments runs; they do not carry personality state and are not used as birth-protocol truth bearers.

2.2 Content Addressing and Immutability

The capsule identifier is the XXHash64 hash of the capsule binary content, computed by `src/starkernel/hash/xxhash64.c`. Because the identifier is derived from the content, any alteration to the payload — including changes to capsule code, CA-key constants, or block layout — produces a different hash. The birth protocol validates this hash before allocating a VM ID, making the content-hash serve as the root-of-trust fingerprint for each capsule.

Capsule immutability has a further consequence for the ACL subsystem: the CA-root public-key placeholder embedded in `ACL.4th` (Block 2066) is part of the hashed content. Replacing the placeholder with a real Ed25519 key necessarily changes the capsule hash, and any such change is detectable by the birth protocol.

2.3 Birth Protocol

The capsule birth protocol executes the following steps in order:

1. Locate the capsule by name in the capsule directory.
2. Validate the content hash (XXHash64); reject the capsule if the computed hash does not match the stored identifier.
3. Allocate a fresh VM ID for the new virtual machine instance.
4. Execute `IDENTITY`: run the capsule code, which establishes the word definitions and initial state specific to this capsule.
5. Execute `PERSONALITY`: load block 1 from the kernel ramdrive (LBN 2048), which supplies the VM's behavioral personality.
6. Log a parity record containing: the VM ID, the capsule hash, and the dictionary hash at the moment personality execution completes.

Parity logging is implemented in `src/starkernel/vm/parity.c`. Each birth and execution event is recorded with sufficient information for an independent system to load the same capsule and compare dictionary hashes. Zero deviation in dictionary hashes across independent runs constitutes the primary evidence for deterministic behavior.

2.4 Capsule Descriptor

Each capsule is described by a `CapsuleDesc` structure. The structure is 64 bytes in size and cache-line aligned to avoid false sharing when the capsule directory is scanned at birth time.

Table 2.2: CapsuleDesc fields

Field	Type	Description
content hash	<code>uint64_t</code>	XXHash64 of capsule binary content
type code	<code>uint8_t</code>	(<code>m</code>), (<code>p</code>), or (<code>e</code>)
name	<code>char[]</code>	Null-terminated capsule name

2.5 Memory Layout

The LithosAnanke kernel partitions VM memory as follows. Block RAM occupies logical block numbers (LBN) 0 through 991, providing 1 MB of dedicated RAM-backed blocks. The kernel ramdrive occupies LBN 2048 through 3071, providing a further 1 MB used for capsule loading. LBN 2048 is the entry point for `init.4th`: when the birth protocol executes Personality, execution begins at this block. The log region occupies LBN 3072 through 5120, providing 2 MB for persistent logging at up to 32,768 lines of 64 bytes each. The full VM memory space is 5 MB in the hosted build (`VM_MEMORY_SIZE`), with the first 2 MB reserved for the dictionary (`DICTIONARY_BLOCKS` = 2048 blocks).

Table 2.3: LithosAnanke LBN layout

LBN range	Size	Purpose
0 – 991	1 MB	Block RAM (dedicated RAM blocks)
2048 – 3071	1 MB	Kernel ramdrive; LBN 2048 = <code>init.4th</code> entry
3072 – 5120	2 MB	Persistent log (32,768 lines × 64 bytes)

2.6 Capsule Files

The `capsules/` directory contains 19 `.4th` source files. These are assembled by `tools/mkcapsule.c` into the binary capsule directory format, which is compiled into the kernel image as `capsule_generated.c`.

Table 2.4: Capsule `.4th` source files

File	Purpose
<code>init.4th</code>	Default Mama VM personality
<code>init-0.4th</code>	Numbered personality variant 0
<code>init-1.4th</code>	Numbered personality variant 1
<code>init-2.4th</code>	Numbered personality variant 2
<code>init-3.4th</code>	Numbered personality variant 3
<code>init-4.4th</code>	Numbered personality variant 4
<code>init-5.4th</code>	Numbered personality variant 5
<code>init-6.4th</code>	Numbered personality variant 6
<code>init-7.4th</code>	Numbered personality variant 7
<code>init-8.4th</code>	Numbered personality variant 8
<code>init-9.4th</code>	Numbered personality variant 9
<code>init-l8-stable.4th</code>	L8 Jacquard: stable mode
<code>init-l8-volatile.4th</code>	L8 Jacquard: volatile mode
<code>init-l8-diverse.4th</code>	L8 Jacquard: diverse mode
<code>init-l8-temporal.4th</code>	L8 Jacquard: temporal mode
<code>init-l8-transition.4th</code>	L8 Jacquard: transition mode
<code>init-l8-omni.4th</code>	L8 Jacquard: omni mode
<code>ACL.4th</code>	Word-level ACL subsystem (self-activating)
<code>zuse.4th</code>	Bootstrap superuser; loaded by <code>ACL.4th</code>

The `mkcapsule` tool (`tools/mkcapsule.c`) reads each `.4th` source file, computes its XXHash64 content hash, and writes a binary capsule directory entry. The resulting `capsule_generated.c` is a C translation unit that, when compiled into the kernel image, provides the capsule directory accessible at boot time without filesystem access.

2.7 Mama Forth Supervisor

Mama Forth is the governing FORTH VM that orchestrates the multi-VM ecosystem. It is itself a FORTH VM, a first-class citizen carrying the same physics instrumentation as every VM it governs. The supervisor does not stand outside the runtime model; it participates in it.

The ecosystem forms a tree. Mama Forth sits at the root. Beneath it, child VMs each carry their own physics state. Beneath each child, block devices carry per-device physics. Mama observes all three layers and makes system-level decisions: when to spawn or kill a child VM, how to allocate block devices to children, how to enforce governance across the ecosystem, and how to respond to anomalous conditions.

The central design insight is that physics metadata becomes a three-tier hierarchy:

- **Word level** — per-word metrics: execution heat, temperature, latency.
- **VM level** — per-VM aggregates: child health, resource usage.
- **Block level** — per-block device metrics: I/O patterns, thermal distribution.

Mama runs with elevated privileges along three axes: observing all word executions in children, all block-device I/O, all child lifecycle events, and all errors; controlling child VM spawn

and kill, block allocation and rebalancing; and coordinating shared block resources and load balancing.

The Mama VM is initialized by the `MAMA_INIT` capsule (type code `(m)`). Exactly one such capsule exists per deployment. Child VMs are born from `PRODUCTION` or `EXPERIMENT` capsules via the birth protocol described in Section [2.3](#).

Chapter 3

Word-Level ACL and Security

StarForth’s word-level access control system combines a thin C infrastructure layer with a pure-FORTH policy capsule, `ACL.4th`. The C layer supplies four `DictEntry` fields for the hot path — a TTL counter, an allow bit, a mode, and a pin — while all policy, all adaptive logic, and all inheritance rules live as colon definitions in `ACL.4th`. A bootstrap superuser, `zuse.4th`, provides the sole authenticated escalation path. The system is implemented through Phase 6; Phase 7 (LithosAnanke parity) and Phase 8 (PKI) remain.

The implementation spans `capsules/ACL.4th`, `capsules/zuse.4th`, `src/word_source/acl_words.c`, and `src/test_runner/modules/acl_words_test.c`.

3.1 C Infrastructure

Four fields in the `DictEntry` structure support the ACL hot path:

Table 3.1: `DictEntry` ACL fields

Field	Type	Description
<code>acl_ttl</code>	counter	Time-to-live; decremented on each execution
<code>acl_allow</code>	bit	Cached allow/deny decision
<code>acl_mode</code>	enum	Enforcement disposition: STRICT, TTL, or PINNED
<code>acl_pinned</code>	bit	One-way ratchet; set-only, never cleared

Two `uint8_t` flags in the `VM` struct control console-level bypasses:

Table 3.2: VM-level ACL flags

Flag	Written by	Effect
<code>emergency_console</code>	C only	Full ACL bypass; fault handler active
<code>zuse_session</code>	C only	Full ACL bypass; superuser authenticated

Both flags are written exclusively by C; no FORTH word sets them directly.

3.1.1 Interpreter Hook: Two-Level Check

The C interpreter reads only two fields per `DictEntry` on each execution:

```
1 if (vm->emergency_console) goto execute; /* 100% bypass --  
   physical access */  
2 if (entry->acl_ttl-- > 0) goto execute; /* TTL good -- single  
   decrement */
```

```

3  acl_recheck(vm, entry);                               /* TTL=0: call FORTH ACL-
    RECHECK */
4  if (!entry->acl_allow) goto reject;

```

The hot path costs one decrement and a branch. The cold path calls `ACL-RECHECK` in `ACL.4th`, which recomputes the adaptive TTL, updates `acl_allow`, and resets the counter. The C side never reasons about policy; it only reads the result.

3.2 Three Enforcement Dispositions

Each dictionary word holds one of three enforcement dispositions:

Table 3.3: ACL enforcement dispositions

Mode	Behavior
STRICT	ACL re-checked on every execution
TTL	ACL cached; re-checked only when the TTL counter reaches zero
PINNED	Mode and decision frozen permanently; a one-way ratchet

A security word selects the mode per word:

```

1  ' MY-WORD ACL-STRICT      \ check every execution
2  ' MY-WORD ACL-TTL-MODE    \ use adaptive TTL
3  ' MY-WORD ACL-PIN        \ freeze -- mode and decision immutable
    forever

```

3.2.1 Adaptive TTL

The TTL is not a fixed value. It is derived from the existing SSM execution physics and continuously updated. The computation defined in `ACL-TTL-COMPUTE` (Block 2063 of `ACL.4th`) is: $TTL = \min(\lfloor \text{heat}/4 \rfloor + \text{ACL-BASE-TTL}, \text{ACL-MAX-TTL})$, where `ACL-BASE-TTL` = 16 and `ACL-MAX-TTL` = 65535.

3.2.2 The Pin Ratchet

`ACL-PIN ( xt - )` is a one-way ratchet. It transitions `STRICT` or `TTL` to `PINNED` and never back. Once pinned, a word's `acl_mode` and `acl_ttl` are immutable within that VM context, and any attempt to alter a pinned word's ACL is a no-op. Kernel primitive words — `EXEC`, `BYE`, and the ACL words themselves — are pinned by `ACL-BOOT` before the first `BIRTH` fires.

3.3 Inheritance at Birth

When a child VM is born, ACL state propagates downward once:

```

1  acl_mode      = parent->acl_mode      \ policy propagates (STRICT or
    TTL)
2  acl_pinned    = 0                     \ always clear -- child owns its
    own lock
3  acl_ttl       = default                \ reset; child has no execution
    history
4  acl_decision  = ALLOW                  \ re-evaluated on first access

```

The pin is contextual, not viral: a parent’s pinned words do not force the child to be pinned. The child inherits the mode as a starting point and may tighten, relax, or re-pin freely. The security lattice flows downward at birth only; afterward each VM’s ACL state is independent, so a compromised child cannot bootstrap its way back to Mama’s pinned ACLs.

ACL-INHERIT (*src dst -*) copies the mode, clears the pin bit, and resets the TTL and decision to defaults for the destination word.

3.4 ACL.4th: Pure-FORTH Policy

ACL.4th is self-activating. `init.4th` needs only one line to enable the entire ACL system:

```
1 S" _ACL.4th" EXEC
```

Internally, Block 2067 of ACL.4th calls ACL-BOOT and then `S" zuse.4th" EXEC`. ACL-BOOT stamps default ACL entries onto every existing dictionary word; after it runs, each subsequent `:` definition receives an ACL entry through the defining-word hook, so no word can exist without one. Leaving the line commented keeps a development build open (no-security mode).

The ACL word set is summarized in Table 3.4.

Table 3.4: ACL.4th word set

Word	Stack	Description
ACL-ENTRY	(<i>xt - xt</i>)	O(1) lookup by XT (identity)
ACL-MODE@	(<i>xt - mode</i>)	Read enforcement mode
ACL-MODE!	(<i>mode xt -</i>)	Set mode (no-op if pinned)
ACL-PINNED?	(<i>xt - flag</i>)	Test pin bit
ACL-PIN	(<i>xt -</i>)	Set pin — one-way, irreversible
ACL-STRICT	(<i>xt -</i>)	Set STRICT mode (no-op if pinned)
ACL-TTL-MODE	(<i>xt -</i>)	Set TTL mode (no-op if pinned)
ACL-TTL@	(<i>xt - n</i>)	Read current TTL counter
ACL-TTL!	(<i>n xt -</i>)	Write TTL counter
ACL-ALLOW@	(<i>xt - flag</i>)	Read cached decision
ACL-ALLOW!	(<i>flag xt -</i>)	Write decision
ACL-INHERIT	(<i>src dst -</i>)	Copy mode; clear pin; reset ttl+decision
ACL-RECHECK	(<i>xt -</i>)	Adaptive TTL recompute; update allow
ACL-TTL-COMPUTE	(<i>xt - ttl</i>)	Compute new TTL from execution heat
ACL-INIT-PRIMITIVES	(<i>-</i>)	Bulk-initialize entries for all words
ACL-BOOT	(<i>-</i>)	Stamp defaults; pin privileged words

3.5 Two-Console Architecture

Two permanent, independent console layers exist at all times.

3.5.1 Emergency Console

The emergency console is always present. Its `emergency_console` flag is a `uint8_t` in the VM struct, written only by C. The flag is checked first in the interpreter hot path, producing a 100% ACL bypass; `acl_recheck()` saves and restores it for re-entrancy protection. The prompt is `ok>`.

The `EMERGENCY_CONSOLE_ENABLED` build flag (default 1) controls whether the interactive fallthrough is compiled in. Setting it to 0 strips the interactive fallthrough entirely, calling

the weak `vm_fault_handler()` symbol instead, which may be overridden for hardware reset, watchdog, or debug-probe integration. `BYE` returns to the emergency console; only `panic` kills the VM.

3.5.2 Zuse Console

The Zuse console is omnipresent and awaits authentication. Its `zuse_session` flag (also a C-only `uint8_t`) grants a full ACL bypass when a Zuse session is active. The prompt is `zuse)ok>`. The two consoles are completely independent: setting one does not affect the other.

3.6 Zuse Superuser

Named for Konrad Zuse, pioneer of programmable computers, Zuse is the bootstrap superuser — the sole entity that can own the Zuse console and mint user credentials. It is defined in `capsules/zuse.4th` and loaded by `ACL.4th` at capsule boot. Zuse’s words are pinned by `ACL-ZUSE-BOOT` at load time.

`ACL-BOOT` runs first, then `S" zuse.4th" EXEC`, both from within `ACL.4th` itself (Block 2067). The current implementation is software-only; no thumbdrive is required. However, `zuse.4th` is a capsule citizen from the first boot, so it carries a content hash and is subject to the same birth-protocol validation as all other capsules.

3.6.1 CA Root

The certificate-authority root is embedded in `ACL.4th` (Block 2066) as the constants `ACL-CA-KEY-L0` and `ACL-CA-KEY-HI`. Because the capsule hash serves as the root-of-trust fingerprint, any change to the CA key changes the capsule hash, and the birth protocol detects the tampering. The key is a zero placeholder until `tools/mkcapsule` embeds the real Ed25519 key at build time.

3.7 Security Model

Security is opt-in through `init.4th`, gated by a single commented-out line:

```
1 \ S" ACL.4th" EXEC
```

Table 3.5: Security activation combinations

ACL.4th	zuse.4th	Result
absent	absent	No security — open development mode
present	absent	ACL enforced; emergency REPL active; Zuse not yet provisioned
present	present	Full lockdown; Zuse owns the Zuse console

Uncommenting the toggle line enables full lockdown. Leaving it commented keeps a development build open.

3.8 ACL System in Kernel Context

Phase 7 (LithosAnanke parity) is the pending work item for the kernel context. Its goals are: verifying that `ACL.4th` loads cleanly in the freestanding kernel environment; running `ACL-BOOT` at kernel boot before the first `BIRTH`; wiring `vm->emergency_console` and `vm->zuse_session` to the kernel REPL and Zuse authentication paths; and achieving three-architecture acceptance (amd64, aarch64, riscv64 booting to `ok>` with ACL active and no regressions), with acceptance logs committed.

3.9 Formal Verification

Phase 6 contributed five Isabelle/HOL proofs of ACL properties:

- `ACL_Pin_Monotone.thy` — the pin bit is set-only; no operation clears it once set.
- `ACL_Inherit_Clears_Pin.thy` — `ACL-INHERIT` always produces an entry with `acl_pinned = 0`, regardless of the source entry’s pin state.
- `ACL_TTL_Bounded.thy` — the TTL counter is bounded above by `ACL-TTL-COMPUTE` output and cannot grow unboundedly.
- `ACL_Emergency_Bypass.thy` — when `emergency_console = 1`, the allow/deny decision is never consulted.
- `ACL_No_Escalation.thy` — a child VM cannot produce a pinned entry with higher privilege than its inherited mode.

These five proofs are part of the 24-theory Isabelle/HOL corpus located in `proof/` (see Volume I for the full proof inventory).

3.10 Implementation Phases

Table 3.6: ACL implementation phase status

Phase	Description	Status
1	C infrastructure: <code>DictEntry</code> fields, interpreter hook, <code>acl_recheck()</code>	Complete
2	<code>ACL.4th</code> : FORTH policy words, XT table, <code>ACL-INIT-PRIMITIVES</code>	Complete
3	<code>capsules/zuse.4th</code> : bootstrap superuser; CA root placeholder	Complete
4	<code>init.4th</code> opt-in toggle	Complete
5	POST tests (800/800)	Complete
6	<code>EMERGENCY_CONSOLE_ENABLED</code> build flag; 5 Isabelle/HOL proofs	Complete
7	LithosAnanke parity: port to kernel context, three-arch acceptance	Pending
8	PKI / thumbdrive: Ed25519 challenge-response; user minting by zuse	Pending

3.11 Future: PKI and Ed25519 (Phase 8)

Phase 8 introduces thumbdrive-based public-key infrastructure. The planned elements are: Ed25519 challenge-response authentication for the Zuse drive, build-time embedding of the real CA key into Block 2066, CA-signed user minting with a home-block image, and a deliberate no-software-recovery policy (loss of the drive requires the administrator to mint a new one via a surviving Zuse instance). All claim-adjacent aspects of Phase 8 are reserved for dictation by Bob and patent counsel.

Chapter 4

Platform Support

StarForth executes across three distinct runtime contexts: the hosted Linux/POSIX environment, the L4Re/Fiasco.OC microkernel, and bare metal via LithosAnanke on three hardware architectures (amd64, aarch64, riscv64). Each context is addressed by a platform abstraction layer that presents a uniform API to the core VM while confining platform-specific code to separate compilation units selected at build time.

4.1 Platform Abstraction Layer

The platform abstraction layer provides portable timing and clock functionality across POSIX systems and the L4Re microkernel, selected by a single Makefile switch. It follows the same vtable pattern as the block I/O subsystem, achieving zero runtime overhead while keeping platform-specific code cleanly separated from the core VM.

The core VM is platform-agnostic and speaks only to `include/platform_time.h`. Behind that header, compile-time selection routes to one of two backends: `src/platform/linux/time.c` using `clock_gettime`, or `src/platform/l4re/time.c` using the RTC server and KIP clock. Builds choose the path explicitly:

```
1 make           # POSIX (default)
2 make L4RE=1     # L4Re/StarshipOS
3 make MINIMAL=1  # minimal embedded, no platform layer
```

4.1.1 Platform API

All timing flows through `platform_time.h`. The host calls `sf_time_init()` once at startup, then reads `sf_monotonic_ns()` for performance measurement (nanoseconds since boot) and `sf_realtime_ns()` for wall-clock time (nanoseconds since the Unix epoch). Additional functions cover clock setting, human-readable timestamp formatting, RTC availability reporting, and unit conversions (`sf_seconds_to_ns`, `sf_ns_to_ms`, and related helpers).

Backend selection occurs at compile time on the `__l4__` define:

```
1 void sf_time_init(void) {
2     #ifdef __l4__
3         sf_time_init_l4re();
4         sf_time_backend = &sf_time_backend_l4re;
5     #else
6         sf_time_backend = &sf_time_backend_posix;
7     #endif
8 }
```

Entries marked “(see note)” are not confirmed in available source material.

Table 4.1: Platform vs. feature support

Feature	Linux/POSIX	L4Re/Fiasco.OC	amd64 (LithosAnanke)	aarch64 / riscv64
Monotonic time	CLOCK_MONOTONIC	KIP clock	TSC + HPET	(see note)
Wall-clock time	CLOCK_REALTIME	RTC server	APIC timer	(see note)
Interactive REPL	yes	yes	yes	yes (QEMU validated)
Block I/O	file + RAM backends	L4Re dataspace	RAM blocks	RAM blocks
Heartbeat thread	yes	yes	M5 timer	(see note)
ACL subsystem	Phases 1–6 complete	(see note)	Phase 7 pending	Phase 7 pending

4.2 Linux / Hosted

The hosted Linux build is the primary development target. The standard build procedure produces an optimized binary:

```

1 make                                # optimized build
2 make fastest                        # ASM + LTO + direct threading
3 make debug                          # symbols enabled
4 ./build/amd64/standard/starforth    # interactive REPL
5 ./build/amd64/standard/starforth --run-tests # run 936+ tests then
    REPL
6 ./build/amd64/standard/starforth -c "1_2_+_._BYE" # inline
    execution

```

The POSIX timing backend uses `CLOCK_MONOTONIC` for high-precision monotonic time, `CLOCK_REALTIME` for wall-clock time, and `localtime/strftime` for timestamp formatting. It always reports an RTC as available.

Key build flags for the hosted target are documented in Chapter 1. The `EMERGENCY_CONSOLE_ENABLED` flag (default 1) is relevant to the hosted build as well as the bare-metal target: setting it to 0 strips the interactive fallback surface, appropriate for production or embedded deployments.

4.3 L4Re / Fiasco.OC

L4Re is a user-level infrastructure layered on the L4 microkernel, providing memory management, task and thread management, inter-process communication, device drivers, and runtime libraries. StarForth installs into the L4Re source tree as `l4/pkg/starforth`.

4.3.1 Package Structure

The package holds: a `Control` metadata file (declaring that the package provides `starforth` and requires `libc`, `libstdc++`, and `l4re-core`), a top-level `Makefile`, a `server/` directory with the L4Re entry point and VM wrapper, a `lib/` directory wrapping the existing `src/` files, the existing `include/`, and an `examples/` client.

4.3.2 Memory Management via Dataspaces

Instead of `malloc`, the L4Re wrapper backs VM memory with a dataspace. Initialization allocates a capability slot, allocates a dataspace of `VM_MEMORY_SIZE`, and maps it into the address space with `L4RE_RM_F_SEARCH_ADDR | L4RE_RM_F_RW`. The mapped region becomes `vm_.memory`. Cleanup detaches the region and frees the capability.

4.3.3 IPC Interface

The server exposes a small opcode protocol: `OP_INTERPRET`, `OP_PUSH`, `OP_POP`, `OP_GET_STATE`, and `OP_RESET`. `OP_INTERPRET` copies an inbound code buffer, runs `vm_interpret`, and returns the VM error status. A matching client wraps these calls, and registration with the name server makes the server reachable under the `starforth` capability.

4.3.4 Multi-VM Architecture on L4Re

A typical deployment runs several FORTH VMs under a root task (Ned), which acts as name server and VM manager. Distinct VMs serve distinct roles: one hosting FORTH tasks, one a request/response server, one an interactive REPL. Ned's `modules.list` wires each VM's capabilities.

4.3.5 L4Re Timing Backend

The L4Re timing backend draws monotonic time directly from the Kernel Info Page clock (`l4_kip_clock_ns(l4re_kip())`) and real time from the RTC server offset added to that clock. It obtains the `"rtc"` capability from the L4Re environment, queries the offset over RPC, and degrades gracefully to epoch time when no RTC server is present. Setting time updates the offset over RPC and requires a write capability.

4.4 Bare Metal: amd64

The amd64 bare-metal path is provided by the LithosAnanke kernel. The architecture-specific layer resides in `src/starkernel/arch/amd64/` and contains:

- `arch.c` — architecture initialization
- `apic.c` — Advanced Programmable Interrupt Controller
- `timer.c` — TSC and HPET timer support
- `interrupts.c` — interrupt management
- `boot.S` — low-level boot assembly
- `isr.S` — interrupt service routine stubs

The UEFI loader (`src/starkernel/boot/uefi_loader.c`) calls `ExitBootServices()` and transfers control to `src/starkernel/kernel_main.c`. Milestone M5 establishes the 100 Hz heartbeat timer using TSC, HPET, and the APIC timer, which drives the adaptive heartbeat subsystem described in Volume I.

Build and QEMU execution:

1	<code>make -f Makefile.starkernel</code>	<code># build UEFI</code>
	<code>image</code>	
2	<code>make -f Makefile.starkernel qemu</code>	<code># run under QEMU/</code>
	<code>OVMF</code>	
3	<code>make -f Makefile.starkernel STARFORTH_ENABLE_VM=1</code>	<code># enable M7 VM</code>
	<code>integration</code>	

Output: `build/amd64/kernel/starkernel_loader.efi` (UEFI PE32+ executable).

4.5 Bare Metal: aarch64 and riscv64

Cross-compilation targets for aarch64 and riscv64 are provided via `Makefile.starkernel`:

```
1 make -f Makefile.starkernel ARCH=aarch64      # aarch64 build  
2 make -f Makefile.starkernel ARCH=riscv64      # riscv64 build  
3 make rpi4-cross                               # Raspberry Pi 4 cross-  
    compile
```

QEMU-based validation is used for all three architectures. Milestones M0 through M5 have been verified under QEMU/OVMF across all three. The `QEMU_BASELINE.log` file captures reference output for regression detection.

4.6 Raspberry Pi Build

```
1 make rpi4-cross      # Cross-compile for Raspberry Pi 4 (aarch64)
```

4.7 Future Platforms

The platform abstraction design anticipates additional backends. Adding a new backend requires: a new `time_<platform>.c` translation unit, its backend vtable entry, a detection branch in `platform_init.c`, and a Makefile flag. Anticipated future targets noted in the platform abstraction design include bare-metal hardware timers (beyond the current amd64/APIC path), WebAssembly via `performance.now()`, RTOSes such as FreeRTOS and Zephyr, and additional microkernels such as seL4.

Chapter 5

L8 Jacquard Mode Selector

5.1 Six Operating Modes

Table 5.1: L8 Jacquard operating modes

Mode	Characteristics
<code>stable</code>	—
<code>volatile</code>	—
<code>diverse</code>	—
<code>temporal</code>	—
<code>transition</code>	—
<code>omni</code>	—

5.2 Attractor Bucket Mechanism

5.3 `ssm__jacquard.c` Implementation

5.4 Jacquard Capsule Variants

Chapter 6

QEMU Testing and Regression Baseline

6.1 Running the Kernel in QEMU

Listing 6.1: Running LithosAnanke in QEMU

```
1 make -f Makefile.starkernel qemu
2 # Requires OVMF UEFI firmware in the OVMF_PATH (see Makefile.
   starkernel)
```

6.2 Regression Baseline

The file `QEMU_BASELINE.log` at the repository root captures the reference QEMU/OVMF output from a known-good LithosAnanke build. Any deviation in a new build's output from this baseline indicates a regression.